Universitatea Politehnica din Timișoara

SHILATE: SOFTWARE-DEFINED VEHICLE REINFORCEMENT LEARNING — FROM SIMULATION TO SDV DEPLOYMENT

Candidate: Given Name FAMILY NAME

Scientific coordinator: Professor Given Name FAMILY NAME

Session: June 2026

ABSTRACT

The proliferation of software-defined vehicle (SDV) platforms has opened an opportunity to combine modern deep reinforcement learning (RL) with real automotive runtime stacks. This thesis presents **SHILATE** — a complete, open-source pipeline that trains a Proximal Policy Optimization (PPO) agent inside a Unity physics simulation and deploys the resulting policy onto an Eclipse Leda SDV stack, bridging the gap between laboratory RL research and realistic vehicle software deployment.

The simulation component is built in Unity 2022 LTS. A single rear-wheel-drive vehicle navigates a circular obstacle course instrumented with a 21-ray LiDAR-style sensor. The training bridge computes a shaped reward signal from progress, collision events, and episode milestones, and communicates with the Python training process exclusively over MQTT, keeping the two subsystems fully decoupled. A dedicated Editor window (*SHILATE Training Controller*) provides real-time health monitoring, metric graphs, and one-click training control without leaving the Unity Editor.

The Python training stack wraps the simulation in a Gymnasium-compliant environment, configures Stable-Baselines3 PPO with a two-layer actor-critic network, and streams structured telemetry back to the Editor. Trained checkpoints are subsequently deployed to the Eclipse Leda SDV image via a containerised MQTT-to-Kuksa bridge and a Velocitas vehicle application that enforces operational limits (overspeed, RPM redline) on the running agent.

Experimental results demonstrate that the PPO agent converges reliably on the obstacle course within 100 000 interaction steps and that the end-to-end deployment chain correctly propagates vehicle telemetry from the simulation through the VSS data layer to the Velocitas safety monitor. The project shows that an integrated sim-to-SDV pipeline is achievable with commodity open-source tooling, and identifies curriculum learning and multi-environment parallelisation as natural extensions for future work.

Keywords: reinforcement learning, software-defined vehicle, Unity simulation, PPO, MQTT, Eclipse Leda, Kuksa, Stable-Baselines3, Gymnasium.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

1. INTRODUCTION

1.1 MOTIVATION AND CONTEXT

The automotive industry is undergoing a fundamental architectural shift. Traditional vehicles are increasingly giving way to software-defined vehicles (SDVs) — platforms in which electronic control units are consolidated onto high-performance compute hardware and the vehicle's behaviour is governed by software layers that can be updated over the air [1]. This transformation enables capabilities that were previously inconceivable: dynamic reconfiguration of driving characteristics, continuous safety-critical updates without a workshop visit, and the integration of machine-learning-based components directly into the vehicle runtime.

At the same time, deep reinforcement learning (RL) has matured into a practical engineering tool. Algorithms such as Proximal Policy Optimization (PPO) [2] achieve human-competitive or superhuman performance in continuous control tasks, with comparatively modest tuning effort. The natural question that arises is: *how can an RL-trained policy be moved, end-to-end, from a training simulation onto an SDV runtime stack in a reproducible and maintainable way?*

This question is not merely academic. Automotive-grade deployment requires that a trained model run inside a standardised software container, interact with vehicle signals through a well-defined data layer, and remain subject to functional-safety monitors that can intervene when the model operates outside its intended envelope. Bridging this gap between laboratory RL and production SDV middleware is the core motivation behind the work presented in this thesis.

1.2 PROBLEM STATEMENT

Existing RL research in the autonomous driving domain concentrates almost exclusively on training performance — reward curves, sample efficiency, and policy robustness [3]. The software engineering challenge of taking a trained checkpoint and deploying it as a first-class citizen of an SDV runtime stack is largely unexplored in the open-source community. Concretely, the following sub-problems remain unaddressed in an integrated, open-source pipeline:

1. **Simulation-to-middleware coupling.** A physics simulator and an SDV middleware stack speak fundamentally different protocols; a communication bridge that is both low-latency and decoupled from either system is needed.
2. **Standardised vehicle-signal representation.** Raw simulator outputs must be mapped onto industry-standard Vehicle Signal Specification (VSS) paths so that the SDV safety layer can interpret them without simulator-specific knowledge.
3. **Operational safety monitoring.** A deployed RL policy may extrapolate outside its training distribution; a lightweight, declarative safety monitor must be able to flag

or override the policy's actions in real time.

4. **Reproducible training workflow.** The entire pipeline — from launching the simulation to saving a deployable checkpoint — must be reproducible from a single control surface without manual orchestration.

1.3 OBJECTIVES

This thesis addresses the above problems through the design, implementation, and evaluation of **SHILATE** (Software-defined-vehicle Hardware-In-the-Loop Autonomous Training Environment). The specific objectives are:

1. Design and implement a Unity-based physics simulation of a rear-wheel-drive vehicle navigating an instrumented obstacle course.
2. Define a shaped reward function that guides a PPO agent to complete laps reliably and safely, and expose a Gymnasium-compliant Python interface over MQTT.
3. Implement a real-time health-monitoring and training-control layer inside the Unity Editor so that a researcher can observe and intervene in a training run without leaving the development environment.
4. Construct an SDV deployment chain using Eclipse Leda, the Kuksa vehicle signal databroker, and a Velocitas vehicle application that enforces operational limits on the running policy.
5. Evaluate the pipeline quantitatively: training convergence, inference lap performance, and end-to-end signal propagation through the SDV stack.

1.4 THESIS STRUCTURE

The remainder of this document is organised as follows. Chapter 2 reviews theoretical background in reinforcement learning, SDV middleware, and communication protocols, and situates SHILATE among related work. Chapter 3 presents the overall system architecture. Chapter 4 describes the Unity simulation in detail. Chapter 5 documents the Python RL training pipeline. Chapter 6 explains the SDV integration through Eclipse Leda. Chapter 7 reports experimental results. Chapter 8 concludes with a summary of contributions and future work.

2. BACKGROUND AND RELATED WORK

2.1 REINFORCEMENT LEARNING FUNDAMENTALS

Reinforcement learning is a computational framework for sequential decision making under uncertainty [4]. An agent interacts with an environment modelled as a Markov Decision Process (MDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the transition kernel, $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor. The agent's goal is to learn a policy $\pi : \mathcal{S} \to \mathcal{A}$ that maximises the expected discounted return $G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$.

Policy gradient methods optimise the policy directly via the gradient of the expected return with respect to policy parameters θ. The fundamental result is the policy gradient theorem:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\nabla_\theta \log \pi_\theta(a|s) \cdot Q^{\pi_\theta}(s, a) \right]. \tag{2.1}$$

Actor-critic architectures maintain both a policy network (actor) and a value network (critic) trained simultaneously; a learned baseline $V_\phi(s)$ reduces gradient variance through the advantage estimate $A(s, a) = Q(s, a) - V_\phi(s)$.

2.2 PROXIMAL POLICY OPTIMIZATION

Proximal Policy Optimization [2] takes the largest possible improvement step on a policy without destabilising training. It replaces Trust Region Policy Optimization's constrained optimisation with a clipped surrogate objective:

$$L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t \left[\min\Big(r_t(\theta)\, \hat{A}_t,\ \mathrm{clip}(r_t(\theta),\, 1{-}\varepsilon,\, 1{+}\varepsilon)\, \hat{A}_t \Big) \right], \tag{2.2}$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\mathrm{old}}}(a_t|s_t)$ is the probability ratio, $\hat{A}_t$ is the estimated advantage, and ε (typically 0.2) is the clipping threshold. The full objective adds a value-function regression term and an entropy bonus:

$$L(\theta) = L^{\mathrm{CLIP}}(\theta) - c_1 L^{\mathrm{VF}}(\theta) + c_2 H[\pi_\theta(\cdot|s_t)]. \tag{2.3}$$

PPO is well suited to the vehicle control domain because its continuous Gaussian policy maps naturally onto a steer/throttle/brake action space, and its on-policy rollout structure integrates cleanly with a wall-clock-rate simulation running in a separate process. The Stable-Baselines3 library [5] provides the production implementation used in this project, validated against reference benchmarks.

2.3 SIMULATION-TO-REAL TRANSFER IN AUTONOMOUS DRIVING

Training autonomous driving policies in simulation rather than on physical hardware is standard practice, motivated by safety, cost, and the ability to reset episodes deterministically [3]. The primary risk is the *sim-to-real gap*: discrepancies between simulated and real sensor

readings, physics, and environmental conditions cause policies that perform well in simulation to degrade on deployment.

Prominent platforms include CARLA [6], an Unreal-Engine-based open-source simulator modelling urban driving at high visual fidelity, and the Unity ML-Agents Toolkit [7], which provides a general-purpose RL training framework tightly integrated with the Unity Editor. SHILATE uses raw Unity physics without ML-Agents for two reasons: it avoids an opaque intermediate API layer, and the training process communicates exclusively through MQTT — the same protocol used by the target SDV stack — so no additional bridge is required for deployment.

2.4 SOFTWARE-DEFINED VEHICLES AND THE VSS DATA MODEL

The COVESA Vehicle Signal Specification (VSS) [8] is a hierarchical, vendor-neutral data model for vehicle signals. VSS paths follow a dotted-name convention, e.g. `Vehicle.Speed` (km/h), `Vehicle.Powertrain.CombustionEngine.Speed` (RPM), and `Vehicle.Chassis.SteeringWheel.Angle` (degrees).

The Eclipse Kuksa databroker [9] implements a VSS-aligned gRPC server acting as the vehicle signal hub. Producers write to VSS paths; consumers subscribe to changes. Eclipse Velocitas [10] builds on Kuksa with a vehicle-application SDK and deployment scaffold. In SHILATE, the Velocitas application monitors the running RL agent for speed and RPM limit violations.

2.5 MQTT AND PUBLISH-SUBSCRIBE MESSAGING

MQTT 3.1.1 [11] is a lightweight publish-subscribe protocol designed for constrained IoT environments. A central broker decouples producers and consumers: a publisher pushes a message to a topic string and all subscribers to that topic receive it with no direct network coupling between the two parties. SHILATE uses QoS 0 (at-most-once delivery) throughout, which minimises latency at the cost of possible message loss — acceptable in a training loop where the next observation replaces the current one within tens of milliseconds. The MQTT protocol is also natively supported by the Eclipse Leda SDV image, which ships Mosquitto as a core service.

2.6 RELATED WORK

A substantial body of work addresses RL for autonomous driving within simulation [3]. OpenAI's `gym-car-racing` [12] provides a top-down racetrack benchmark but offers no path to real vehicle deployment. CARLA-based RL studies [6] achieve high visual realism but operate as closed systems with no integration to SDV middleware. The Unity ML-Agents toolkit [7] enables RL training within Unity but couples the Python trainer tightly to Unity's side-channel API, making it difficult to reuse the same environment against a physical vehicle.

From the SDV side, the Eclipse Kuksa and Velocitas communities have demonstrated containerised vehicle applications that read VSS signals and enforce policies, but without an RL training pipeline attached. SHILATE contributes the missing link: a training-to-deployment

chain that is agnostic to both the simulation engine and the SDV stack as long as both speak MQTT and VSS.

3. SYSTEM ARCHITECTURE

3.1 HIGH-LEVEL OVERVIEW

SHILATE is structured as three loosely coupled tiers, each running as an independent process or container and communicating exclusively through a Mosquitto MQTT broker. This design ensures that any tier can be replaced or upgraded without modifying the others, and that the same communication fabric used during training is available unchanged on the deployment target.

Tier 1 — Unity Simulation. A Unity 2022 LTS Editor instance runs the vehicle physics, the obstacle course, and the sensor model. It publishes observations and reward signals to MQTT and consumes control commands from the same broker. The Unity process also hosts the *Training Controller* Editor window, which provides a developer-facing UI for health monitoring and training control.

Tier 2 — Python RL Trainer. A Python process wraps the MQTT interface in a Gymnasium-compliant environment [13], instantiates a Stable-Baselines3 PPO agent [5], and runs the training loop. It emits structured telemetry back through MQTT so the Editor window can display live metrics. After training, the same Python process can be re-invoked in inference mode with a saved checkpoint.

Tier 3 — Eclipse Leda SDV Stack. The Eclipse Leda SDV image [1] runs on QEMU or physical hardware. It hosts a Mosquitto broker, a Kuksa databroker, an `mqtt-kuksa-feeder` container that maps MQTT telemetry to VSS paths, and a Velocitas vehicle application that monitors operational limits.

3.2 COMMUNICATION BACKBONE

All inter-tier communication passes through a single Mosquitto MQTT 3.1.1 broker [11]. The topic namespace uses the fixed prefix `env0/`, reflecting the single-environment design. The complete topic set is listed in Table 3.1.

Table 3.1: MQTT topic map for the SHILATE training loop.

Topic	Direction	Payload
`env0/leda/control/steer`	Python→Unity	float $[-1, 1]$
`env0/leda/control/throttle`	Python→Unity	float $[0, 1]$
`env0/leda/control/brake`	Python→Unity	float $[0, 1]$
`env0/leda/control/reset`	Python→Unity	"1"
`env0/vehicle/speed`	Unity→Python	float km/h
`env0/vehicle/training/obs`	Unity→Python	JSON array
`env0/vehicle/training/reward`	Unity→Python	float
`env0/vehicle/training/done`	Unity→Python	bool
`env0/vehicle/training/episode_end`	Unity→Editor	JSON object
`env0/vehicle/training/heartbeat`	Unity→Editor	JSON (1 Hz)

QoS 0 (at-most-once delivery) is used throughout. Given that observations arrive at 10 Hz and each new observation makes the previous one irrelevant, occasional message loss is harmless and the absence of acknowledgement handshakes reduces loop latency significantly compared to QoS 1 or 2.

3.3 TRAINING DATA-FLOW

Each training step follows a synchronised request-response pattern over MQTT. The Python environment calls `step(action)`, which:

1. Publishes steer, throttle, and brake to the three control topics.
2. Blocks on a threading event until Unity publishes a new `training/obs` message, or a 5-second timeout elapses.
3. Reads the co-published reward and done values.
4. Returns `(obs, reward, terminated, truncated, info)` to Stable-Baselines3.

This cycle runs at the Unity physics rate (approximately 50 Hz wall-clock), throttled by the 10 Hz sensor publication interval. The PPO rollout buffer accumulates 2048 transitions before a gradient update.

3.4 CONFIGURATION MANAGEMENT

Configuration is split across three locations:

- **`config.json`** (repository root): holds `model.ray_count` (default 9 in the root copy; the simulation runs 21 rays). Read by `config_loader.py` with fallback.
- **`TrainingSettings.asset`** (Unity ScriptableObject): persists training knobs configured through the Editor window — MQTT host/port, virtual-environment path, script paths, PPO hyperparameters, checkpoint save location, and optional resume-model path. Survives Editor restarts without an on-disk config file.

- **CLI flags** (`train.py`): hyperparameters from TrainingSettings are forwarded as command-line arguments when `PythonProcessManager` spawns the trainer, keeping the Python process stateless with respect to the Editor.

3.5 DEPLOYMENT ARCHITECTURE ON ECLIPSE LEDA

When a trained checkpoint is deployed onto the Leda image, three containers run simultaneously:

1. **`leda-controller`** (inference mode): loaded with the `best_model.zip` checkpoint; subscribes to `env0/vehicle/training/obs` and publishes control commands.
2. **`mqtt-kuksa-feeder`**: bridges `env0/vehicle/#` telemetry topics to VSS paths in the Kuksa databroker via gRPC.
3. **`velocitas-app`**: subscribes to `Vehicle.Speed` and engine RPM in the Kuksa databroker and publishes alerts to `leda/command/alert` when thresholds are exceeded.

Each container is described by a Kanto manifest (`kanto-manifest.json`) specifying the OCI image reference, environment variables, and restart policy. The provisioning script `leda/provision-device.sh` copies manifests to the device and triggers Kanto to pull and start the containers.

4. UNITY SIMULATION DESIGN

4.1 SCENE CONSTRUCTION

The simulation scene is constructed programmatically at runtime rather than authored in the Unity Editor. `RuntimeSceneBuilder.cs` runs on scene load and checks whether the required game objects are present; if absent, it invokes `CarFactory` to spawn the full vehicle hierarchy and `SpawnObstacleCourse` to instantiate the track. This approach guarantees a well-defined initial state regardless of how the Editor project was last saved.

The obstacle course is a circular ring generated by `ObstacleCourse.cs`. The ring is bounded by two concentric cylindrical walls: inner radius 25 m, outer radius 40 m, giving a track width of 15 m. Twelve obstacle objects are distributed at equal angular intervals. Wall height is 3 m, sufficient to be detected by raycasts but low enough to avoid occluding the follow camera. The vehicle is spawned at position $(32.5,\ 0.5,\ 0)$ — on the track centreline facing the initial direction of travel.

4.2 VEHICLE PHYSICS MODEL

The vehicle is a rear-wheel-drive configuration assembled by `CarFactory.cs`. The chassis mass is 1500 kg. Four `WheelCollider` components provide the physics contact model using Unity's [14] built-in Pacejka-inspired tyre model. Key parameters are listed in Table 4.1.

Table 4.1: Key vehicle physics parameters.

Parameter	**Value**	**Unit**
Chassis mass	1500	kg
Maximum motor torque	450	Nm
Maximum steer angle	35	degrees
Final drive ratio	9.0	–
Maximum brake torque	3000	Nm
Regenerative brake torque	150	Nm
Maximum speed	150	km/h

`VehicleController.cs` translates the three-component action vector $[a_{\text{steer}}, a_{\text{throttle}}, a_{\text{brake}}]$ received from `RemoteDriveInput.cs` into WheelCollider motor torques and steer angles. A creep behaviour applies a small constant torque in drive gear when both throttle and brake inputs are near zero, mirroring a real automatic-transmission vehicle.

4.3 PERCEPTION LAYER

The vehicle's sole sensor is a LiDAR-inspired raycast fan implemented in `RaycastSensor.cs`. Twenty-one rays are cast in the vehicle's horizontal plane, spanning a $120°$ arc centred on the forward axis, with a range of 50 m. Each ray returns a normalised distance $d_i \in [0, 1]$ where 0 indicates contact at the origin and 1 indicates no contact within range. The sensor publishes the full ray array to `env0/vehicle/training/obs` at 10 Hz.

The complete observation vector received by Python is:

$$\mathbf{o} = \left[d_0, \ldots, d_{N-1}, \frac{v}{v_{\max}}, \frac{\delta + 1}{2} \right] \in \mathbb{R}^{N+2}, \tag{4.1}$$

where N is the configured ray count, v is current speed (km/h), $v_{\max} = 150$ km/h, and δ is steer angle normalised to $[-1, 1]$. All elements of $\mathbf{o}$ lie in $[0, 1]$, simplifying network initialisation.

4.4 REWARD FUNCTION

The per-step reward is:

$$r_t = r_{\text{progress}} + r_{\text{collision}} + r_{\text{milestone}}, \tag{4.2}$$

where:

- $r_{\text{progress}} = 2 \cdot \Delta p_t$ — signed change in track checkpoints since the last step.
- $r_{\text{collision}} = -5$ on any step where the vehicle reports a collision contact (applied once per event).
- $r_{\text{milestone}} = +50$ on full lap completion; $+25$ on first half-turn of the course (at most once per episode each).

An episode terminates on lap completion, prolonged standstill, or after the 60-second wall-clock timeout. Timeout terminations are reported as `truncated=True` so that Stable-Baselines3 bootstraps the value estimate rather than treating the transition as a terminal failure.

4.5 MQTT BRIDGE IN UNITY

Unity does not ship a production-grade MQTT client. SHILATE includes a minimal custom `MqttClient.cs` that implements the MQTT 3.1.1 wire protocol [11] natively in C# without external dependencies. The client handles `CONNECT`, `PUBLISH`, `SUBSCRIBE`, `PINGREQ/PINGRESP`, and `DISCONNECT` packet types with a 30-second keep-alive.

`LedaBroker.cs` provides the application-facing API, configured via `LedaBroker.Configure(host, port, "env0")` at startup. It publishes vehicle telemetry at 10 Hz and receives control commands. A Python heartbeat timeout (5-second threshold) detects when the trainer has stopped sending commands and holds the vehicle stationary until reconnection.

4.6 TRAINING CONTROLLER EDITOR WINDOW

The *SHILATE Training Controller* (`SHILATE → Training Controller`) is an EditorWindow that eliminates manual terminal management during training. Its layout from

top to bottom comprises:

1. **Toolbar**: training duration and MQTT connection indicator.
2. **Health banner**: colour-coded strip showing the current state (*Idle*, *Healthy*, *Warning*, *Critical*, *Disconnected*) with explicit problem text.
3. **Snapshot row**: episode count, last-episode reward, 10-episode rolling mean, and heartbeat counter.
4. **Settings panel** (collapsible): all TrainingSettings fields exposed as GUI controls.
5. **Metric graphs**: scrolling plots of episode reward mean, policy gradient loss, value loss, and KL divergence.
6. **Issue log**: timestamped list of every health-state transition for the session.
7. **Controls**: *Start Training* / *Run Model* / *Stop*.

The *Start Training* button enters Play mode programmatically and, once the scene is loaded and the MQTT connection is confirmed, launches the Python training script with arguments from TrainingSettings. The health evaluator monitors five independent failure modes: MQTT disconnection, Python process crash, observation gap exceeding 5 seconds, heartbeat/reward gap exceeding 60 seconds, and reward collapse (more than 50% drop over a 10-episode window).

5. PYTHON RL TRAINING PIPELINE

5.1 GYMNASIUM ENVIRONMENT WRAPPER

The Python training pipeline is built on the Gymnasium API [13], which defines a standard interface (`reset()`, `step()`, `observation_space`, `action_space`) that any Stable-Baselines3 algorithm can consume without modification. The wrapper class `ShilateEnv(gym.Env)` in `environment.py` translates between this interface and the MQTT broker.

The observation space mirrors (4.1):

$$\mathcal{O} = \mathrm{Box}(0,\ 1,\ \mathrm{shape}{=}(N{+}2,),\ \mathrm{dtype}{=}\texttt{float32}).$$

The action space is:

$$\mathcal{A} = \mathrm{Box}([-1,0,0],\ [1,1,1],\ \mathrm{shape}{=}(3,),\ \mathrm{dtype}{=}\texttt{float32}),$$

corresponding to steer $\in [-1,1]$, throttle $\in [0,1]$, and brake $\in [0,1]$.

`reset()` sends a reset command to Unity, then blocks until either an `obs` message arrives or the 5-second `OBS_TIMEOUT` elapses. `step(action)` publishes the three action components to their control topics, then blocks on a threading event until a new `obs` arrives alongside co-published `reward` and `done` values. The threading-event approach avoids busy polling and keeps CPU usage low while waiting.

5.2 PPO AGENT CONFIGURATION

The PPO agent is instantiated from Stable-Baselines3 [5], which uses PyTorch [15] as its deep-learning backend. The environment is wrapped in a one-slot `DummyVecEnv` as required by the SB3 API. The neural network uses separate actor and critic branches:

$$\pi:\ \mathbb{R}^{N+2} \xrightarrow{256} \mathbb{R}^{128} \to \mathbb{R}^{|\mathcal{A}|}, \qquad V:\ \mathbb{R}^{N+2} \xrightarrow{256} \mathbb{R}^{128} \to \mathbb{R}.$$

Default PPO hyperparameters are listed in Table 5.1.

Table 5.1: Default PPO hyperparameters.

Parameter	**Default**	**CLI flag**
Total timesteps	100,000	`--total-timesteps`
Learning rate	3×10^{-4}	`--learning-rate`
Rollout steps n	2048	`--n-steps`
Batch size	64	`--batch-size`
Clip ε	0.2	SB3 default
Discount γ	0.99	SB3 default
GAE λ	0.95	SB3 default

5.3 TRAINING LOOP AND HEALTH TELEMETRY

The training loop is driven by `PPO.learn(total_timesteps)`. A custom `HealthCallback(BaseCallback)` is attached and invoked at the end of each rollout. It inspects policy gradient loss, value loss, and KL divergence for NaN or infinite values and emits a `SHILATE-HEALTH nan_in_losses` marker to stdout if detected, causing the Editor banner to transition to *Critical*.

All metrics are transmitted to the Unity Editor through two channels:

1. **SB3 tabular output.** Stable-Baselines3 prints a formatted table to stdout every n steps. `TrainingMetricsParser.cs` parses the rows `rollout/ep_rew_mean`, `train/policy_gradient_loss`, `train/value_loss`, and `train/approx_kl`.
2. **Structured markers.** The `emit()` helper writes lines of the form `SHILATE-METRIC key=value` and `SHILATE-HEALTH signal` to stdout, enabling the Editor parser to extract specific values without fragile regex matching.

Model checkpoints are saved every approximately 10,000 timesteps; `best_model.zip` is maintained by an `EvalCallback`.

5.4 INFERENCE MODE

`ai_driver.py` loads a saved `.zip` checkpoint and runs the policy in a closed-loop inference loop. Observations are constructed from the same MQTT source as the training environment. The policy is evaluated with `deterministic=True`, disabling Gaussian exploration noise. Episode telemetry (speed, steer, throttle, brake) is logged every 50 steps. The script runs a fixed episode count (`--episodes`) or loops indefinitely (`--episodes 0`).

5.5 CROSS-PLATFORM PROCESS MANAGEMENT

`PythonProcessManager.cs` spawns and terminates the Python process from within the Unity Editor. On Windows it invokes `cmd.exe /c` with the `Scripts\activate` venv path; on Linux/macOS it uses `bash -c` with `bin/activate`. Graceful shutdown sends SIGTERM (or the Windows equivalent Ctrl+C event) to the process group, allowing the Python process to flush TensorBoard logs and save the final checkpoint before Play mode exits.

6. SDV INTEGRATION WITH ECLIPSE LEDA

6.1 ECLIPSE LEDA OVERVIEW

Eclipse Leda [1] is an SDV distribution built on the Yocto Project. It packages a curated set of Eclipse SDV middleware components — Mosquitto MQTT broker, Kuksa databroker, Kanto container manager, and OpenTelemetry collector — into a bootable image that runs on QEMU (x86-64) or on supported ARM hardware such as the Raspberry Pi 4. Its primary purpose is to serve as a reference platform for developing and validating SDV software stacks before deploying to a real vehicle ECU.

SHILATE targets the QEMU image (`sdv-image-all-qemux86-64.wic.qcow2`) included in the repository. The `leda/run-leda.sh` script launches the image with a bridged network interface so that the host machine and the QEMU guest share a Mosquitto broker at `127.0.0.1:1883`.

6.2 MQTT-TO-KUKSA BRIDGE

The `mqtt-kuksa-feeder` container subscribes to the `vehicle/#` wildcard on the Mosquitto broker and forwards each received message to the Kuksa databroker [9] as a VSS path write via gRPC. The topic-to-VSS mapping is shown in Table 6.1.

Table 6.1: MQTT topic to VSS path mapping.

MQTT topic	VSS path
`vehicle/speed`	`Vehicle.Speed`
`vehicle/rpm`	`Vehicle.Powertrain.CombustionEngine.Speed`
`vehicle/steering`	`Vehicle.Chassis.SteeringWheel.Angle`
`vehicle/brake`	`Vehicle.Chassis.Brake.PedalPosition`
`vehicle/throttle`	`Vehicle.OBD.ThrottlePosition`

After the feeder writes a signal, any subscriber — including the Velocitas vehicle application — is notified with the new value within the same event loop tick.

6.3 VELOCITAS VEHICLE APPLICATION

The Velocitas vehicle application [10] implements the operational safety monitor for the RL agent. It subscribes to five VSS signals via the Kuksa databroker, logs a telemetry summary every 5 seconds, and enforces two hard limits:

- **Overspeed**: if `Vehicle.Speed` exceeds 120 km/h, an alert is published to `leda/command/alert` with payload `{"type":"overspeed","value":<speed>}`.
- **RPM redline**: if engine RPM exceeds 6500, an analogous alert is published.

Alerts are currently informational; the architecture is designed so that the safety monitor can be upgraded to a command-injection path without modifying any other component.

6.4 CONTAINER DEPLOYMENT

All three service containers (`leda-controller`, `mqtt-kuksa-feeder`, `velocitas-app`) ship with Dockerfiles producing minimal Python 3.11 slim images. Each is described by a Kanto container manifest (`kanto-manifest.json`) specifying the OCI image reference, environment variables (`MQTT_HOST`, `MQTT_PORT`, `MODE`, `MODEL_PATH`), and restart policy. Provisioning is automated by `leda/provision-device.sh`, which uses SSH to copy manifests to the Leda device and invoke the Kanto CLI to pull and start the containers.

6.5 LEDA TELEMETRY BROKER

In addition to MQTT, the Leda image runs a lightweight telemetry broker implemented in Rust. The broker listens on TCP port 7878 and acts as a multiplexing relay: any client that connects and sends telemetry data has that data forwarded to all other connected clients. This enables tools such as `netcat` or custom dashboards to observe the live telemetry stream without subscribing to individual MQTT topics.

7. EXPERIMENTS AND RESULTS

7.1 EXPERIMENTAL SETUP

All training experiments were conducted on a workstation running Windows 11 with the Unity Editor invoked via WSL2 for the Python process. The environment is summarised in Table 7.1.

Table 7.1: Experimental environment.

Component	**Specification**
CPU	Intel Core i7-12700H (14 cores)
RAM	32 GB DDR5
GPU	NVIDIA RTX 3070 Ti (8 GB)
OS	Windows 11 / WSL2 Ubuntu 22.04
Unity	2022.3 LTS
Python	3.10.12
Stable-Baselines3	2.3.0
PyTorch	2.1.0
Gymnasium	0.29.1
MQTT broker	Mosquitto 2.0.18

The Mosquitto broker ran on the host at `127.0.0.1:1883`. Unity physics ran at the default fixed time-step of 0.02 s (50 Hz) with `Time.timeScale = 1` (real time). The Python process executed in a WSL2 terminal and communicated with Unity exclusively over the loopback interface.

7.2 TRAINING CONVERGENCE

A baseline training run of 100,000 timesteps was conducted using the default hyperparameters from Table 5.1. The episode reward mean, computed over a rolling 10-episode window as reported by Stable-Baselines3, is the primary convergence indicator.

At the start of training the agent explores nearly uniformly, producing episode rewards in the range $[-5, +10]$. By approximately 20,000 timesteps the agent consistently avoids the inner wall and begins accumulating progress rewards, with the rolling mean rising to $[20, 40]$. The half-turn bonus of $+25$ is reliably triggered from around 30,000 timesteps. By 80,000 timesteps the agent completes full laps in the majority of episodes and the rolling mean stabilises in the $[60, 80]$ range. Table 7.2 summarises milestones across three independent seeds.

Table 7.2: Training convergence milestones (mean over three seeds).

Milestone	Timestep	Std. dev.
First collision-free episode	12,400	$\pm$2100
Half-turn bonus reliably triggered	31,000	$\pm$3800
First full lap completed	48,500	$\pm$5200
Rolling mean ≥ 60	76,000	$\pm$6900

7.3 REWARD SHAPING ABLATION

To evaluate the contribution of the half-turn milestone bonus, a second training run was performed with $c_h = 0$ (all other parameters unchanged). Without the half-turn bonus, the first complete lap was delayed by approximately 18,000 timesteps on average and the rolling mean reward at 100,000 steps was approximately 12 points lower. This confirms that the intermediate milestone provides a useful progress signal early in training.

A further ablation with reduced collision penalty ($c_c = -1$ instead of -5) produced an agent that completed laps but drove closer to the inner wall, indicating that the stronger penalty is necessary to encourage a safety margin.

7.4 INFERENCE EVALUATION

The `best_model.zip` checkpoint was evaluated over 50 episodes using `ai_driver.py`. Results are summarised in Table 7.3.

Table 7.3: Inference evaluation over 50 episodes.

Metric	Value
Lap completion rate	82% (41/50)
Mean episode reward	63.4
Mean lap duration	38.2 s
Mean speed (completed laps)	47.6 km/h
Collisions per episode	0.6
Timeout episodes	9 (18%)

The 18% of episodes that ended in timeout typically corresponded to the agent entering a slow-speed oscillation near an obstacle, suggesting that the current reward shaping does not sufficiently penalise stalling. This is identified as a direction for future improvement.

7.5 HEALTH MONITORING VALIDATION

The health monitoring system was validated by inducing controlled failures:

- **Observation timeout.** The Unity simulation was paused mid-episode. The health evaluator correctly transitioned to *Critical* after 5 seconds with the message “No observation received for 5 s”.

- **Python crash.** The Python process was killed with SIGKILL. The Editor detected the non-zero exit code and displayed “Python process exited unexpectedly (exit code 137)”.
- **MQTT disconnect.** Mosquitto was stopped. The EditorMqttListener raised a connection-lost event within approximately 35 seconds (one keep-alive period) and the banner transitioned to *Disconnected*.

All three failure modes were correctly detected and displayed without manual intervention.

7.6 END-TO-END SDV INTEGRATION TEST

The full deployment chain was validated on the QEMU-hosted Leda image. With all three containers running, the following checks were performed:

1. A Kuksa `GetValue` call for `Vehicle.Speed` returned the current simulator speed within 100 ms of it being published to MQTT, confirming correct feeder operation.
2. When the Python agent applied maximum throttle for 5 seconds, the Velocitas application published an overspeed alert to `leda/command/alert`, confirming end-to-end signal propagation from the simulation through the VSS layer to the safety monitor.
3. The alert was received by a test `mosquitto_sub` subscriber on the host machine, confirming that the outbound MQTT path from the Leda image to the host network was functional.

8. CONCLUSIONS

8.1 SUMMARY OF CONTRIBUTIONS

This thesis presented SHILATE, an open-source pipeline that trains a deep reinforcement learning agent inside a Unity physics simulation and deploys the resulting policy onto an Eclipse Leda SDV runtime stack. The main contributions are as follows.

An integrated simulation environment for SDV-targeted RL. A Unity 2022 LTS scene was constructed programmatically, comprising a realistic rear-wheel-drive vehicle physics model, a 21-ray LiDAR-style sensor, and a circular obstacle course. The environment publishes observations and consumes control commands through MQTT at 10 Hz, using the same protocol and topic namespace as the deployment target, so that no protocol translation is required when moving from training to deployment.

A shaped reward function with quantified milestone bonuses. A four-term reward signal was designed and ablated: a continuous progress reward, a collision penalty, a half-turn milestone bonus, and a lap-completion bonus. Ablation experiments confirmed that the intermediate milestone bonus accelerates convergence by approximately 18,000 timesteps and that the magnitude of the collision penalty directly controls the agent's safety margin.

A Gymnasium/SB3 training pipeline with live Editor integration. The MQTT interface is wrapped in a Gymnasium-compliant environment, allowing any Stable-Baselines3 algorithm to be substituted without modification. A Unity Editor window provides one-click training control, real-time health monitoring across five independently evaluated failure modes, and live metric graphs.

A complete SDV deployment chain. A containerised MQTT-to-Kuksa bridge maps simulator telemetry to COVESA VSS paths, and a Velocitas vehicle application enforces operational limits on the running RL policy. The entire chain was validated end-to-end on a QEMU-hosted Eclipse Leda image.

8.2 LIMITATIONS

Single training environment. SHILATE supports exactly one Unity Editor instance at real time (`Time.timeScale = 1`). Training throughput is therefore limited by wall-clock simulation speed.

No sim-to-real gap mitigation. The sensor model, vehicle physics, and obstacle geometry are all abstractions. No domain randomisation or adaptation mechanism is included, so a policy trained in SHILATE is unlikely to transfer directly to a physical vehicle without further fine-tuning.

Informational-only safety monitor. The Velocitas application publishes alerts but does not currently override the agent's control commands. The deployment chain provides monitoring rather than enforcement.

Limited action space. The current action space does not include gear selection or parking-brake inputs, restricting the range of manoeuvres available to the agent.

8.3 FUTURE WORK

Multi-environment parallelisation. Replacing the single Unity Editor instance with headless builds or container-based simulation instances would dramatically increase sample efficiency and enable distributed RL algorithms such as IMPALA.

Curriculum learning. Automatically varying the obstacle configuration as the agent improves would implement a difficulty curriculum and is expected to address the stalling behaviour observed in the inference evaluation.

Physical hardware deployment. Validating the deployment chain on a physical SDV platform — such as a CAN-bus-instrumented RC vehicle running Eclipse Leda on embedded hardware — would provide direct evidence of the pipeline's applicability to real automotive systems.

Safety monitor upgrade. Extending the Velocitas application to publish override control commands when a limit is exceeded would close the loop from detection to intervention.

Richer observation space. Adding front-facing camera images or occupancy-grid representations would bring the perception model closer to production ADAS systems, at the cost of increased training complexity.

9. BIBLIOGRAPHY

[1] Eclipse Leda Project, *Eclipse leda: SDV distribution documentation*, Accessed June 2026, 2023. [Online]. Available: https://eclipse-leda.github.io/leda/.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017. [Online]. Available: https://arxiv.org/abs/1707.06347.

[3] B. R. Kiran, I. Sobh, V. Talpaert, *et al.*, "Deep reinforcement learning for autonomous driving: A survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 6, pp. 4909–4926, 2021. DOI: 10.1109/TITS.2021.3054625.

[4] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018. [Online]. Available: http://incompleteideas.net/book/the-book-2nd.html.

[5] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: http://jmlr.org/papers/v22/20-1364.html.

[6] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proceedings of the 1st Annual Conference on Robot Learning (CoRL)*, 2017, pp. 1–16. [Online]. Available: https://arxiv.org/abs/1711.03938.

[7] A. Juliani, V.-P. Berges, E. Teng, *et al.*, "Unity: A general platform for intelligent agents," *arXiv preprint arXiv:1809.02627*, 2020. [Online]. Available: https://arxiv.org/abs/1809.02627.

[8] COVESA – Connected Vehicle Systems Alliance, *Vehicle signal specification (VSS)*, Accessed June 2026, 2023. [Online]. Available: https://covesa.github.io/vehicle_signal_specification/.

[9] Eclipse Kuksa Project, *Eclipse Kuksa.VAL documentation*, Accessed June 2026, 2024. [Online]. Available: https://eclipse-kuksa.github.io/kuksa.val/.

[10] Eclipse Velocitas Project, *Eclipse velocitas documentation*, Accessed June 2026, 2024. [Online]. Available: https://eclipse-velocitas.github.io/velocitas-docs/.

[11] OASIS, "MQTT version 3.1.1," OASIS Standard, Tech. Rep., 2014. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html.

[12] G. Brockman, V. Cheung, L. Pettersson, *et al.*, "OpenAI gym," *arXiv preprint arXiv:1606.01540*, 2016. [Online]. Available: https://arxiv.org/abs/1606.01540.

[13] M. Towers, A. Kwiatkowski, J. K. Terry, *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv preprint arXiv:2407.17032*, 2024. [Online]. Available: https://arxiv.org/abs/2407.17032.

[14] Unity Technologies, *Unity manual — unity 2022 LTS*, Accessed June 2026, 2022. [Online]. Available: https://docs.unity3d.com/2022.3/Documentation/Manual/.

[15] A. Paszke, S. Gross, F. Massa, *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 8024–8035. [Online]. Available: https://arxiv.org/abs/1912.01703.

DECLARAȚIE DE AUTENTICITATE A LUCRĂRII DE FINALIZARE A STUDIILOR *

Subsemnatul **ION POPESCU**, legitimat cu **CI** seria **TZ** nr. **100772**, CNP **5000102355779**, autorul lucrării **TITLUL LUCRĂRII DE FINALIZARE A STUDIILOR** elaborată în vederea susținerii examenului de finalizare a studiilor de **LICENȚĂ** organizat de către Facultatea **AUTOMATICĂ ȘI CALCULATOARE** din cadrul Universității Politehnica Timişoara, sesiunea **IUNIE** a anului universitar **2022**, coordonator **dr.ing. MIHAI IONESCU**, luând în considerare conținutul art. 34 din *Regulamentul privind organizarea și desfășurarea examenelor de licență/diplomă și disertație*, aprobat prin HS nr. 109/14.05.2020 și cunoscând faptul că în cazul constatării ulterioare a unor declarații false, voi suporta sancțiunea administrativă prevăzută de art. 146 din Legea nr. 1/2011 – legea educației naționale și anume anularea diplomei de studii, declar pe proprie răspundere, că:

- această lucrare este rezultatul propriei activități intelectuale,
- lucrarea nu conține texte, date sau elemente de grafică din alte lucrări sau din alte surse fără ca acestea să nu fie citate, inclusiv situația în care sursa o reprezintă o altă lucrare/alte lucrări ale subsemnatului.
- sursele bibliografice au fost folosite cu respectarea legislației române şi a convențiilor internaționale privind drepturile de autor.
- această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de licență/diplomă/disertație.

Timișoara,

Data Semnătura

_____ _____

* Declarația se completează „de mână” şi se inserează în lucrarea de finalizare a studiilor, la sfârşitul acesteia, ca parte integrantă.